



Determining Destroy Sets in Large Neighborhood Search by Generative Flow Networks

Maria Bresich¹(✉) , Jingyi Peng¹, Günther R. Raidl¹ ,
and Steffen Limmer²

¹ Institute of Logic and Computation, TU Wien, Vienna, Austria
{mbresich,raidl}@ac.tuwien.ac.at, e12402257@student.tuwien.ac.at

² Honda Research Institute Europe GmbH, Offenbach/Main, Germany
steffen.limmer@honda-ri.de

Abstract. Large Neighborhood Search (LNS) is an often applied metaheuristic for solving challenging combinatorial optimization problems. Typically, it relies on the scheme of iteratively removing parts of a solution by one or more *destroy* operators and cleverly augmenting the solution again by *repair* operators. Destroy operators are usually handcrafted and often a mixture of uniform random selection and problem-specific heuristics, and their choice can dramatically impact the performance. Inspired by the recent success of Generative Flow Networks (GFlowNets), we propose their usage to compile promising destroy sets. These generative deep learning models stand out in that their goal is not to learn a single high-reward structure but to produce diverse structures according to a complex probability density function. As considering many different but promising destroy sets is fundamental in LNS, we believe GFlowNets to be well suited for this purpose. We study an intentionally generic and simple LNS framework and apply it to three hard graph problems. The only problem-specific part is the repair operator that is realized by basic iterated random construction heuristics. When applying the best GFlowNet model, the effectiveness of the LNS is on average increased by 6.52% in terms of online performance compared to a corresponding baseline LNS. Moreover, a comparison to integer linear programming models solved by Gurobi shows that they are clearly dominated by our GFlowNet-guided LNS on the considered hard benchmark instances with a mean relative improvement of 20.73% in terms of final solution quality.

Keywords: Generative Flow Networks · Large Neighborhood Search

1 Introduction

Large neighborhood search (LNS) [2,27] is a highly popular metaheuristic for addressing challenging combinatorial optimization problems (COPs), with applications in vehicle routing, scheduling, timetabling, and location planning, see

e.g., [5, 14, 20, 21, 26]. The key idea is to explore large neighborhoods of an incumbent solution instead of just small local changes as in local search. Most variants rely on the iterative *destroy and repair* scheme: a current solution is partially destroyed by removing a subset of its elements (or un-assigning corresponding decision variables), and then repaired either by heuristic or exact methods. Acceptance can be based on non-deterioration or more sophisticated criteria, and the process is repeated until a termination criterion is met. The performance of such an LNS depends heavily on the choice of its destroy and repair operators. A substantial variability of destroy sets across iterations is crucial to enable diversification and a good coverage of the search space. Destroy operators often use simple uniform random selection or are problem-tailored by domain experts.

The manual design and tuning of destroy operators, however, is time-consuming, and an “ideal” destroy operation may also depend more generally on the current state of the whole search. This limitation has fueled learning-based approaches for selecting the elements to destroy within an LNS [1, 6, 8, 9, 23, 28, 29]. Such neural LNS approaches typically use graph neural networks (GNNs) trained either in supervised fashion on labeled data obtained by time-expensive calculations or by reinforcement learning (RL) [30]. All these approaches suffer from what is known as *mode collapse* in generative machine learning [11]: For a given current solution, the learned model tends to repeatedly produce the same (or a similar) destroy set—more generally called *mode*—despite the existence of usually many different highly promising modes. This can hinder escaping local optima if this destroy set fails to improve the solution. The proposed methods therefore add random noise or use problem-specific tricks to increase destroy set variability, but this does not really address the issue of mode collapse. Ideally, the model should be able to produce diverse high quality destroy sets covering all or at least many promising modes on its own.

A relatively new family of generative models that is explicitly designed to produce structures according to a complex probability density function and thus avoid mode collapse is that of Generative Flow Networks (GFlowNets) [3, 4]. Recent applications to COPs [17, 33–35] have demonstrated their ability to sample high-quality solutions with high diversity. We therefore propose and investigate the usage of GFlowNets for constructing promising destroy sets within an LNS framework that we intentionally keep simple and generic to support diverse COPs. The only problem-specific components are the repair operator, realized via a simple iterated random construction heuristic, and the GFlowNet’s underlying neural network. GFlowNets operate in an auto-regressive manner following a Markov decision process (MDP), which we tailor for use within the LNS. We intentionally minimize feature usage and investigate different reward assignments to enable effective training. Our GFlowNets use graph isomorphism networks and are trained offline on representative unlabeled problem instances. We demonstrate the benefits and generality of the approach through extensive experiments on a hard variant of the maximum independent set (MIS), the maximum cut (MAXCUT), and the densest k -subgraph (DkS) problems across three graph types. These COPs are classical benchmarks and many real-world prob-

lems can be modeled similarly as graph problems. We compare variants of the GFlowNet-guided LNS (GFN-LNS) against a classical baseline LNS and integer linear programming (ILP) models solved by Gurobi. GFN-LNS is shown to mostly outperform both baselines in terms of final solution quality and convergence speed (online performance) despite the relative simplicity of the setup, with the GFlowNet overhead justified by the gains. Additional tests on smaller and larger instances than those used for training further highlight the generalization capabilities of GFN-LNS. In summary, our main contributions are:

- A generic GFlowNet-guided LNS framework with model-based destroy set generation applicable to diverse COPs.
- A problem-agnostic MDP with effective reward assignments for training.
- A comprehensive evaluation of the approach on three hard graph problems across different graph types, demonstrating advantages over both baseline LNS and ILP approaches as well as strong generalization capabilities.

The next section provides necessary preliminaries and related work and specifies the COPs for benchmarking. Section 3 describes our GFlowNet-guided LNS framework, including the baseline LNS approach and detailing the MDP formulation and training procedure. Section 4 presents and discusses experimental results, and Sect. 5 concludes the paper, also outlining promising future work.

2 Preliminaries

We now provide background information on GFlowNets and related LNS approaches and introduce the benchmark problems we consider.

2.1 Generative Flow Networks

This relatively new family of stochastic generative deep learning models has been introduced in 2021 by Bengio et al. [3]. While bearing similarities with RL, they are trained to sample composite objects x of some object family $\mathcal{X}$ with probabilities proportional to a given non-negative reward function $R(x) \geq 0$. In contrast, standard RL seeks to maximize expected cumulative reward and thus tends to concentrate probability mass on a single high-reward output.

Formal Definitions. A GFlowNet operates on a directed acyclic graph (DAG) $\mathcal{G} = (\mathcal{S}, \mathcal{A})$, where $\mathcal{S}$ is a finite set of *states* and $\mathcal{A} \subseteq \mathcal{S} \times \mathcal{S}$ is a set of *transitions* (actions). A unique *source state* $s_0 \in \mathcal{S}$ has no incoming edges, while *terminal states* $x \in \mathcal{X} \subseteq \mathcal{S}$ have no outgoing edges. A trajectory $\tau = (s_0, s_1, \dots, s_{n_\tau} = x)$ is a path in $\mathcal{G}$ from the source to a terminal state and represents the sequential construction of a specific object x . The model maintains a *forward policy* $P_F(s' \mid s)$, which is a probability distribution over the successor states s' that can be reached from each non-terminal state s , as well as a *backward policy* $P_B(s \mid s')$ indicating the probability that state s is reached from one of its predecessor states s' . A non-negative *flow function* $F : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ assigns an amount of flow to each state, with terminal states satisfying $F(x) = R(x)$ and the source

node having the total flow $F(s_0) = Z = \sum_{x \in \mathcal{X}} R(x)$. Flow conservation requires that for every interior state $s \in S \setminus \{s_0\} \setminus \mathcal{X}$

$$\sum_{s': (s, s') \in \mathcal{A}} F(s) P_F(s' | s) = \sum_{s'': (s'', s) \in \mathcal{A}} F(s'') P_F(s | s'') \quad (1)$$

holds. A GFlowNet satisfying this condition provably samples each terminal state x with probability $P(x) = R(x)/Z$ [3].

Training Objectives. In practice, P_F , P_B , and F are parameterized by neural networks that take a state s as input and output transition probabilities or log-flows. GFlowNets are then trained by considering many trajectories in conjunction with their rewards and trying to establish the above conditions across these trajectories by updating the models' parameters using classical stochastic gradient descent. This can be done in different ways, and similarly to [34], we consider the following established loss functions as alternatives.

The *detailed balance (DB) loss* [4] aims at achieving $F(s) P_F(s' | s) = F(s') P_B(s | s')$ at each transition (s, s') of a considered trajectory. For numerical stability, this is expressed in log-domain and deviations are squared, yielding

$$\mathcal{L}_{\text{DB}}(s, s') = (\log F(s) + \log P_F(s' | s) - \log F(s') - \log P_B(s | s'))^2. \quad (2)$$

The *forward looking (FL) loss* [24] is a variation of the DB loss in which *intermediate rewards* are also considered, with the hope that they lead to faster convergence. It is defined as

$$\mathcal{L}_{\text{FL}}(s, s') = (\log F(s) + \log P_F(s' | s) - \log F(s') - \log P_B(s | s') + \mathcal{E}(s \rightarrow s'))^2, \quad (3)$$

where $\mathcal{E}(s \rightarrow s')$ is a log-reward contribution given for each transition in a trajectory; it is assumed that these contributions are additive and add up to $\log R(x)$.

The *trajectory balance (TB) loss* [22] avoids a general model for state flows $F(s)$ but just learns the total flow Z as scalar in addition to the forward and backward policies and tries to achieve consistency across complete trajectories by aiming for $Z \cdot \prod_{t=1}^{n_\tau} P_F(s_t | s_{t-1}) = R(x) \cdot \prod_{t=1}^{n_\tau} P_B(s_{t-1} | s_t)$. Expressing this in log-domain and squaring deviations yields

$$\mathcal{L}_{\text{TB}}(\tau) = \left(\log Z + \sum_{t=1}^{n_\tau} \log P_F(s_t | s_{t-1}) - \log R(x) - \sum_{t=1}^{n_\tau} \log P_B(s_{t-1} | s_t) \right)^2. \quad (4)$$

Training with TB may converge faster than with DB because the credit-assignment signal is distributed over an entire trajectory rather than individual edges [22]. On the downside, TB in general exhibits higher variance and can be less stable than DB.

Training Regimes. GFlowNets support both *off-policy* and *on-policy* training. For off-policy, trajectories are collected under a behavior policy that differs from P_F and are stored in a replay buffer for repeated use. In the on-policy setting,

trajectories are generated by rolling out the current P_F . This flexibility and the general robustness of training are notable advantages over many RL algorithms, where off-policy training is not always possible or requires significant algorithmic overhead. Section 3.3 will describe our specific training procedure in the context of the LNS framework, which essentially is on-policy with small adaptations.

GFlowNets for Combinatorial Optimization. Zhang et al. [34] demonstrated that GFlowNets equipped with GNNs can generate high-quality heuristic solutions for NP-hard COPs on graphs, including the MIS, MAXCUT, and minimum dominating set problems. By training a GFlowNet to construct solutions autoregressively, with rewards derived from the objective value, the authors showed that the learned sampler produces competitive solutions while maintaining a high level of diversity. Building on this line of work, GFlowNets have since been applied to further combinatorial domains, including scheduling [33] and vehicle routing [35]. Extensions of GFlowNets to stochastic environments [15, 25] are also relevant in the context of combinatorial optimization.

2.2 Machine Learning Based Destroy Operators in LNS

Related to our work, deep learning techniques have recently already been used for selecting which parts of a current solution to destroy within an LNS framework. Song et al. [28] proposed a general RL framework for learning which variables to fix or branch on within an ILP-guided LNS, while Addanki et al. [1] and Chen et al. [6] train (recurrent) GNNs to learn problem-specific destroy heuristics for integer programs and vehicle routing, respectively. Sonnerat et al. [29] and Oberweger et al. [23] train generative models based on GNNs via imitation learning to approximate an expert policy that optimally selects variables for re-optimization in LNS. A more advanced contrastive learning based approach is proposed by [12], while Cherrered et al. [7] employ graph attention networks to integrate node- and route-level features for more impactful removals. All these approaches potentially suffer from the issue of mode collapse, although they apply different tricks to increase the variability of the generated destroy sets. Moreover, they all rely on the node-wise logits from a single model pass to select a destroy set. While this is computationally efficient, it also implies that different modes are naively mixed, and specific dependencies between elements are not well captured. These weaknesses are directly addressed by the GFlowNet-based approach we propose. Another promising direction is the use of denoising diffusion models to generate destroy sets, as proposed in [8]. Like GFlowNets, they can also capture many modes, but a large number of necessary inferencing steps for the iterative denoising may lead to high computational overhead.

2.3 Considered Benchmark Problems

To assess the generality of our approach, evaluation is done on three different NP-hard COPs on graphs that are classical benchmarks in combinatorial optimization and cover different types of constraints and objective structures: a

MIS-BNB	MAXCUT
$\begin{aligned} \max \quad & \sum_{v \in V} x_v \\ \text{s.t.} \quad & x_u + x_v \leq 1 \quad (u, v) \in E \\ & b_u \geq x_v \quad u \in V, v \in N(u) \\ & \sum_{u \in V} b_u \leq B \\ & x_v, b_v \in \{0, 1\} \quad v \in V \end{aligned}$	$\begin{aligned} \max \quad & \sum_{(u,v) \in E} y_{u,v} \\ \text{s.t.} \quad & y_{u,v} \leq x_u + x_v \quad (u, v) \in E \\ & y_{u,v} \leq 2 - (x_u + x_v) \quad (u, v) \in E \\ & x_v \in \{0, 1\} \quad v \in V \\ & y_{u,v} \in \{0, 1\} \quad (u, v) \in E \end{aligned}$
DkS	
$\begin{aligned} \max \quad & \sum_{(u,v) \in E} y_{u,v} \\ \text{s.t.} \quad & y_{u,v} \leq x_u, y_{u,v} \leq x_v \quad (u, v) \in E \\ & \sum_{v \in V} x_v = k \\ & x_v \in \{0, 1\} \quad v \in V \\ & y_{u,v} \in \{0, 1\} \quad (u, v) \in E \end{aligned}$	

Fig. 1. ILP models for the three considered graph problems.

variant of the MIS, the MAXCUT, and the DkS problem [10, 16, 19]. All problems are defined on a simple undirected graph $G = (V, E)$ with $n = |V|$ nodes and $m = |E|$ edges. We briefly explain the problems in the following, and corresponding basic ILP models, which we also use as a baseline for comparison in the experiments, are given in Fig. 1. In all these models, binary decision variables x_v indicate with value one that node $v \in V$ is part of solution set S .

Maximum Independent Set with Budgeted Neighborhood Blocking (MIS-BNB). An independent set (IS) is a subset $S \subseteq V$ where no two vertices are adjacent. The classical MIS problem seeks to maximize $|S|$ and is well solvable by modern ILP solvers and specialized algorithms such as KaMIS [18] on many practically relevant instance classes. We therefore consider a more challenging variant with a *blocking budget* $B \geq 0$ similar to the large secluded IS problem [31]. A vertex $u \notin S$ is *blocked* if it has at least one neighbor in S , and the goal is to find a MIS such that the total number of blocked vertices does not exceed B . The global budget constraint yields a weak LP relaxation that makes this variant substantially harder for ILP solvers as well as heuristics. In the ILP model, additional binary variables b_v are used to indicate blocked vertices.

Maximum Cut (MAXCUT). A cut of a graph is a subset of edges whose removal disconnects the graph into two components V_1 and $V_2 = V \setminus V_1$. The MAXCUT problem seeks a set V_1 that maximizes the number of edges in the cut, i.e., between V_1 and V_2 . In the ILP model, additional binary variables $y_{u,v}$ indicate whether edge (u, v) is in the cut.

Densest k -Subgraph (DkS). Given a parameter $k \leq n$, the DkS problem seeks a vertex set $S \subseteq V$ with $|S| = k$ that maximizes the number of edges in the induced subgraph $G[S]$ [19]. In the ILP model, additional binary variables $y_{u,v}$ indicate whether both endpoints of edge (u, v) are selected in S .

3 GFlowNet-Guided LNS (GFN-LNS)

We first describe the generic LNS framework that serves as our baseline and which will subsequently be enhanced by replacing the destroy operator with a learned GFlowNet. The Markov decision process (MDP) formulation for training the GFlowNet is then detailed, including the specific reward design that we use. The training process is explained, and we also provide details on the GFlowNet architecture and implementation.

3.1 Large Neighborhood Search Framework

We employ a standard LNS framework as described before, and in our setting, the decision variables correspond to nodes in the problem graph $G = (V, E)$. The destroy operator selects a subset $D \subseteq S$ of nodes from the current solution set $S \subseteq V$ and removes them, yielding a partial solution. The repair operator augments the remaining solution with additional nodes to obtain a new complete feasible solution, which is accepted if it is not worse than the current solution. As a baseline, we use a destroy operator that selects a fixed number of nodes from S uniformly at random. For the repair operator, we employ a simple and also rather generic *iterated random repair* strategy. In each iteration, this method considers all nodes not currently in the solution set in random order and inserts each node provided that the problem-specific feasibility constraints are not violated and the objective value does not decrease. The iterated random repair executes this random fill procedure r times independently, each time starting from the same destroyed partial solution, and returns the best solution obtained across all r repetitions. This repetition reduces the variance inherent in the randomized construction and increases the probability of finding a high-quality completion. The initial solution for the LNS is constructed by applying random repair to the empty solution.

3.2 GFlowNet-Based Destroy Set Selection

The main goal of this work is to improve upon the basic random destroy operator by selecting the subsets of nodes for removal more meaningfully and targeted. We train and apply a GFlowNet to choose in each iteration the elements to be destroyed one-by-one in an auto-regressive manner until a predefined number of nodes is reached. In this study, this number is fixed to keep the setup simple and comparable to the baseline. Note that auto-regressive construction of the destroy set allows the GFlowNet to capture complex dependencies between nodes, which we also see as a key advantage over approaches that select destroy sets based on the node-wise logits of a single GNN inference. The downside, however, is the increased computational overhead, which we also analyze in the experiments.

MDP Formulation and Reward Design. A GFlowNet trajectory corresponds to one isolated LNS-iteration consisting of a single destroy and repair operation as well as the acceptance decision for the candidate solution. The GFlowNet’s DAG

for construction of a destroy set is modelled as an MDP with states, actions, transitions, and a reward function. In this context, a *state* $s_t = (G, S, D_t)$ consists of the problem instance graph G , the current solution $S \subseteq V$, and the so far compiled (partial) destroy set $D_t \subseteq S$. Optionally, we provide information on earlier applied destroy sets as additional destroy history feature EMA^{sel} : Specifically, we store for each node an exponential moving average (EMA) of whether a node was selected for removal (value one) or not (value zero) in previous LNS iterations. The state representation and feature set are intentionally minimalistic and generic as we want to investigate the GFlowNet’s abilities under raw basic conditions. However, providing further features either on global aspects of the search, such as the iteration number and aspects on convergence, or problem-specific features, such as centrality measures or the number of selected neighbors in S , would likely further improve the GFlowNet’s performance.

The initial (source) state $s_0 = (G, S, D_0 = \emptyset, \text{EMA}^{\text{sel}})$ corresponds to the current solution of the LNS iteration and an empty destroy set. A terminal state is reached when $|D_t| = d$ for a predefined number of nodes d and is associated with the solution obtained from the repair operation after performing the destroy and the subsequent acceptance decision. An *action* $a_t = v \in S \setminus D_t$ represents the selection of a node v from the current solution that has not been picked yet for removal, and *transitions* always correspond to the update of the state by adding the selected node to the destroy set, i.e., $D_{t+1} = D_t \cup \{v\}$. Non-terminal successor states are deterministic, whereas terminal successor states are stochastic due to the randomized repair operator. This stochasticity is mitigated by employing iterated random repair which reduces the variance.

The *reward function* is a mapping $R : \mathcal{X} \rightarrow \mathbb{R}_{>0}$ of terminal states to positive numbers, defined as $R(x) = e^{-\mathcal{E}(S)/T}$ with $\mathcal{E}(s)$ denoting a problem specific energy function on the solution S associated with terminal state s and T denoting a temperature parameter [34]. If the candidate solution after the repair step is not accepted, the reward is set to one, which corresponds to an energy of zero. Otherwise, we base the reward and energy calculation on the improvement achieved by the repair step, i.e., the difference in objective value $\Delta = \text{obj}(S) - \text{obj}(S')$ between the repaired solution S and the initial solution S' of the iteration, where obj denotes the problem-specific objective function. In LNS, it is important to consider that finding large improvements over low quality (initial) solutions early on may be trivial whereas a small improvement over an already high quality solution near the end may be much harder to find and more valuable. Thus, the value of an improvement depends on the quality of the initial solution but also on the instance size, and rewards should ideally be normalized to be comparable across different instances and sizes. Using the negated raw improvement $-\Delta$ as energy does not reflect this context awareness and could be an insufficient base for reward calculation. As we train on homogeneous instances of equal size, a fixed global constant can be determined in advance as a baseline b for the objective value of high quality solutions. We thus propose a normalization procedure based on the remaining gap of the trajectory’s initial solution to this baseline, $\text{gap} = b - \text{obj}(S')$, with a lower bound of one to prevent shrinking rewards

when $\text{obj}(S') > b$. The normalized improvement is also capped at one to treat improvements exceeding the gap equally, i.e., $\Delta_{\text{norm}} = \min(\Delta / \max(\text{gap}, 1), 1)$. To further ensure that even small improvements receive a noticeable reward and to provide a learning signal that clearly distinguishes between improving and non-improving actions, we extend the energy calculation to a hybrid approach using a weighted sum of the gap-based normalized improvement and a binary indicator for whether the solution was improved, such that

$$-\mathcal{E}(S) = \begin{cases} 0 & \text{if } S \text{ is not accepted,} \\ \alpha \cdot \mathbb{I}[\Delta > 0] + (1 - \alpha) \cdot \Delta_{\text{norm}} & \text{if } S \text{ is accepted} \end{cases} \quad (5)$$

with $\alpha \in [0, 1]$ denoting a weight parameter. The case of $\alpha = 1$ corresponds to relying on the binary improvement indicator alone, to which we will also compare the hybrid approach in the evaluation.

3.3 GFlowNet Training

The training of the GFlowNet is done in an on-policy manner where trajectories are iteratively generated by running our GFN-LNS on always different training instances. In each iteration, the destroy set elements are sampled according to probabilities provided by the current forward policy P_F . Depending on the employed training objective, training samples are derived either from complete trajectories (when using TB) or by taking all transitions from each trajectory (for DB and FL) of a batch. As an LNS run usually performs a large number of iterations, it is expected that only a relatively small fraction of iterations will yield improvements, thus resulting in an excess of unsuccessful trajectories. To mitigate this data imbalance, we reduce the number of unsuccessful trajectories from an LNS run by first removing all trajectories after the last successful LNS iteration as we cannot be sure that any further improvement is possible. If sequences of unsuccessful iterations exceed a given length threshold γ (in our experiments $\gamma = 10$), we further only keep γ randomly sampled ones from these.

We consider the loss functions TB, DB, and FL already explained in Sect. 2.1. In case of the FL loss, we linearly interpolate from zero to the terminal reward for determining the corresponding log-reward contributions $\mathcal{E}(s \rightarrow s')$ for each trajectory. Before training, we run our baseline LNS (cf. Sect. 3.1) on a set of representative problem instances for two purposes: The initial states of successful trajectories are collected to be used as independent validation set for regularly evaluating the model during training. We also compute the mean objective value (obj) and standard deviation ($\sigma(\text{obj})$) of the best solutions found across all runs and derive the baseline b for the energy calculation as $b = \text{obj} + \sigma(\text{obj})$.

The GFlowNet is trained for a specified number of iterations. In each iteration, a new batch of instances is created and training trajectories are generated via on-policy rollouts of GFN-LNS. This coupling between data generation and the model to be trained may enable better and more fitting training data over time as well as a better trained model in the end. In each training iteration, a predefined number of update steps is performed on mini-batches of the collected

trajectories employing one of the loss functions. To improve learning stability and convergence speed, we track an EMA of all seen rewards and center the rewards of each batch by subtracting this EMA. At regular iteration intervals, the GFlowNet is evaluated on the validation set and metrics such as the averages of loss, objective value, and rewards are measured. The training always completes all iterations and the final model to be used is selected based on the best validation performance across iterations, similarly to an early stopping approach.

GFlowNet Configuration. For the neural networks within the GFlowNet, we use an architecture similar to the one proposed by Zhang et al. [34] and refer to this work for more details. For the forward policy P_F and state flow function F when using DB or FL loss, we employ independent graph isomorphism networks (GINs) [32] with five convolutional layers, a hidden dimension of 256, and sum aggregation. The backward policy is defined as a uniform distribution over all possible feasible predecessor states. As in [34], we use the Adam optimizer with a learning rate of 10^{-3} for all learnable functions P_F , F , and Z . After preliminary tests, the temperature parameter T for reward calculation was set to 0.1. Training is done for 3000 iterations with 20 update steps each, using a batch size of 32, and we evaluate the model every 10 iterations.

4 Experiments

All solution approaches were implemented in Julia 1.12.5 and we used Gurobi 13.0.0¹ in single-threaded mode as ILP solver with a memory limit of 10 GB. The experiments were performed on a server with an NVIDIA A40 GPU with 45 GB memory and single cores of 4.05 GHz AMD EPYC 9274F processors. For training and benchmark instances, we consider random graphs of type Erdős-Rényi (ER), Random k -Regular (RR), and Barabási-Albert (BA) graphs, which are commonly used in the literature and cover diverse structural properties, and apply Julia’s `Graphs.jl` package to create corresponding instances. We train a separate GFlowNet model for each combination of problem and graph type which takes between 6 and 47 h wall-clock time on the given hardware depending on the problem-graph configuration and especially the employed loss function, with TB loss being the slowest because of worse sample efficiency. Training graphs have $n = 300$ nodes and the following type-specific parameters: an edge probability $p_e = 0.1$ for ER, a degree $k = 4$ for RR (increased to $k = 20$ for DkS to get harder instances), and an attachment parameter $m = 10$ for BA. The values of problem-specific parameters, i.e., the blocking budget B for MIS-BNB and the subgraph size k for DkS, depend on the graph type and are chosen to yield instances that are generally hard to solve. The considered values are as follows: for MIS-BNB, we use $B = \lfloor \frac{2}{3}n \rfloor$ for ER, $B = \lfloor \frac{1}{3}n \rfloor$ for RR, and $B = \lfloor \frac{1}{2}n \rfloor$ for BA; for DkS, we use $k = 0.2n$ for ER and RR and $k = 0.4n$ for BA. The same

¹ <https://www.gurobi.com>.

configurations are used for generating 100 independent benchmark instances of each problem and graph type combination for evaluation.

We compare multiple variants of our GFN-LNS employing the different training objectives TB, DB, and FL to two baselines: the ILP models solved by Gurobi and the classic LNS with a uniform random selection of destroy sets, which we refer to as ILP and LNS-rand, respectively. Since finding high quality solutions fast is challenging but also essential in practice, we set a time limit of three seconds for all methods and instances. This is also to keep training efforts reasonable, but in general, consideration of longer run times would be possible. For the LNS-based approaches, a fixed destroy set size $d = 5$ is used in training as well as testing and across all configurations, and the number r of repetitions for the iterated random repair is varied depending on the problem to achieve a good balance between solution quality and run time; these values were determined in preliminary experiments. For testing, we use $r = 2000$ for MIS-BNB, $r = 1000$ for MAXCUT, and $r = 8000$ for DkS, and 1/10 of these iterations are used for training to speed up the process. Parameters specific to the GFlowNet training include the weight parameter $\alpha = 0.1$ for the energy calculation, and parameters for the EMA calculations for the history feature and the reward centering, which are set to 0.9 and 0.99. Again, these values were determined in preliminary experiments.

4.1 Results

In the following evaluations, we employ as main metrics the final solution quality, indicated by the mean objective value ($\overline{\text{obj}}$) and standard deviation ($\sigma(\text{obj})$) of the best solutions found across all 100 runs on the test instances, and the online performance in terms of the area under the curve (AUC) of the achieved objective value over time. In tables, bold values indicate best values, and also the average number of executed iterations is given as I^{total} for the LNS approaches. Shaded areas in the plots always illustrate the standard deviation across runs.

Table 1 compares the performance of our three main GFN-LNS variants employing the different loss functions to each other and the baselines across all combinations of problems and graph types. We observe that the ILPs yield the highest variance in all cases and perform notably worse than all LNS approaches, e.g., TB-based GFN-LNS achieves on average 20.73% better objective values. Even when increasing the time limit for Gurobi tenfold in additional experiments—given as ILP-30 in italics in the table and running out of competition due to the extended time limit—, it is still outperformed by GFN-LNS within three seconds in 2/3 of the cases. This confirms the practical hardness of the considered instances and motivates the usage of heuristic methods. Next, we observe that in 7/9 cases, GFN-LNS beats LNS-rand regarding final solution quality, with an additional win when considering the AUC, in terms of which GFN-LNS with TB loss performs on average 6.52% better. The first result shows that the benefit of the learned destroy sets is large enough to justify the time overhead induced by inferencing the models. This overhead is significant and decreases the number of executed iterations on average by 38% compared to LNS-rand, which

Table 1. Results on benchmark instances for all problems (MIS-BNB, MAXCUT, DkS) across three different types of graphs (ER, RR, and BA) with 300 nodes.

Prob	Method	ER				RR				BA			
		obj	$\sigma(\text{obj})$	AUC[%]	I^{total}	obj	$\sigma(\text{obj})$	AUC[%]	I^{total}	obj	$\sigma(\text{obj})$	AUC[%]	I^{total}
MIS-BNB	ILP	12.00	1.18	—	—	52.66	1.66	—	—	34.72	3.55	—	—
	<i>ILP-30</i>	<i>18.50</i>	<i>1.30</i>	—	—	<i>53.76</i>	<i>1.32</i>	—	—	<i>42.14</i>	<i>2.57</i>	—	—
	LNS-rand	20.65	0.83	88.27	241	53.49	1.37	90.54	339	42.58	2.61	92.03	254
	GFN-LNS-TB	20.60	0.85	90.45	155	53.23	1.32	89.33	185	42.73	2.56	94.98	157
	GFN-LNS-DB	20.53	0.95	87.99	157	53.45	1.38	90.12	185	42.36	2.54	90.92	157
	GFN-LNS-FL	20.36	0.97	83.22	154	53.70	1.35	91.19	184	42.17	2.50	88.37	160
MAXCUT	ILP	2470.87	41.20	—	—	489.40	10.00	—	—	1659.57	23.08	—	—
	<i>ILP-30</i>	<i>2470.87</i>	<i>41.20</i>	—	—	<i>513.30</i>	<i>3.28</i>	—	—	<i>1665.01</i>	<i>26.34</i>	—	—
	LNS-rand	2787.07	36.43	80.89	239	507.06	3.89	85.14	508	1867.25	11.12	84.36	300
	GFN-LNS-TB	2799.79	36.97	89.95	157	505.14	4.55	84.51	205	1868.48	13.47	86.53	175
	GFN-LNS-DB	2797.27	36.63	90.42	157	505.34	4.85	79.08	204	1864.40	12.51	82.42	177
	GFN-LNS-FL	2785.30	35.53	81.17	158	505.70	3.94	81.88	205	1864.99	11.95	81.44	177
DkS	ILP	300.63	11.48	—	—	178.06	7.62	—	—	977.32	68.74	—	—
	<i>ILP-30</i>	<i>329.84</i>	<i>11.09</i>	—	—	<i>221.63</i>	<i>6.08</i>	—	—	<i>1103.59</i>	<i>3.31</i>	—	—
	LNS-rand	339.22	8.25	80.28	170	224.32	4.34	70.41	182	1067.39	6.49	81.62	179
	GFN-LNS-TB	348.21	10.26	86.05	121	236.85	4.00	83.21	124	1102.45	3.09	94.63	126
	GFN-LNS-DB	348.26	8.74	86.24	120	245.38	4.27	87.36	124	1100.86	3.30	92.40	125
	GFN-LNS-FL	339.87	8.71	80.95	120	226.72	4.78	71.32	124	1100.61	3.31	93.46	125

is usually detrimental to LNS performance but the improvements enabled by the better selection of destroy sets make up for it. That larger improvements are found quicker by GFN-LNS is also reflected in the second observation regarding the better online performance, which is exemplarily illustrated in Fig. 2. The only case where no GFN-LNS variant outperforms LNS-rand is for MAXCUT on RR graphs, which we explain by their regular structure and consequently a lack of what a GFlowNet can learn to exploit. We further find that for MIS-BNB in general, there are only small performance differences among the LNS approaches and the variance is low across all methods. Regarding the other problem-graph combinations, GFN-LNS with TB loss performs best, closely followed by the variant employing DB loss, which is especially successful for DkS. Using FL loss often results in worse performance, which indicates that our simple intermediate reward calculation by linear interpolation is not informative enough.

Considering these results, we restrict further analyses to the GFN-LNS variant with TB loss, and to MAXCUT and DkS on ER and BA graphs. We evaluate the destroy set diversity exemplarily for MAXCUT-ER by computing the mean pairwise Jaccard distance $d_J(D', D'')$ [13] for destroy sets D' and D'' from different iterations within stagnation periods. These are sequences of iterations without improvement where the destroy sets stem from the same solution. Averaging this distance across all stagnation periods and runs yields an overall diversity measure with values close to one (zero) indicating high (low) diversity. Both LNS-rand and GFN-LNS-TB exhibit high values of 0.9829 and 0.9755, respectively, confirming the diversity of the produced destroy sets. In ablation studies, we investigate different components of our GFN-LNS, i.e., the design of the energy

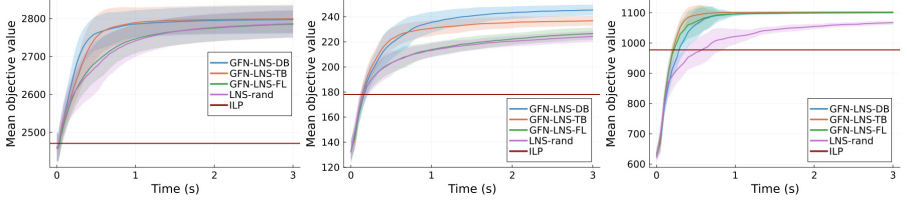


Fig. 2. Performance over time for MAXCUT on ER (left), DkS on RR (center), and DkS on BA (right). The line for ILP represents the final objective value.

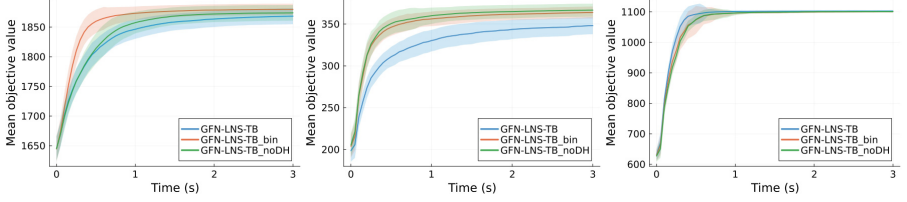


Fig. 3. Performance over time for different GFN-LNS variants for MAXCUT on BA (left), DkS on ER (center), and DkS on BA (right). Usage of the binary indicator for energy calculation is indicated by “bin” and “noDH” denotes the removal of the destroy history feature EMA^{sel} .

calculation and the usage of the destroy history feature EMA^{sel} , to get insights on their impact on the performance. The destroy set diversity is barely impacted by the binary approach for energy calculation with $d_J = 0.9754$ whereas removal of EMA^{sel} results in a slightly lower value of 0.9696. Complete results are too extensive to show here, but further interesting findings are illustrated in Fig. 3. According to pairwise one-sided Wilcoxon signed-rank tests (WSRT) with a threshold $p \leq 0.05$, removing EMA^{sel} significantly improves the performance for DkS-ER and MAXCUT-BA (but not in other cases), and for these as well as MAXCUT-ER, we observe that using the binary approach leads to significantly better performance than the hybrid normalization especially in the beginning. We believe this is due to the binary indicator providing a more robust and more distinct learning signal for improving actions and that the finer-grained consideration of objective gains can sometimes also be more distracting. Both results clearly underline that our GFlowNets are able to effectively learn without having to rely on more advanced reward design and features as often needed in other learning approaches. Nonetheless, for cases such as DkS-BA where results are close and variance is low, these enhancements seem to be slightly beneficial to capture important information about the search process.

Generalization. To test the generalization capabilities of GFN-LNS, we consider smaller and larger instances than those used for training, i.e., graphs with 200, 400, and 500 nodes. To maintain the structural properties of the graphs,

Table 2. Generalization results on instances of size 200 to 500 for different problem and graph type combinations. For each case, the GFlowNet model performing best on $n = 300$ is used which differ in whether the destroy history feature (Dest. Hist.) is used (y) or not (n) and which energy calculation approach is used ($\alpha = 0.1$ for the hybrid and $\alpha = 1$ for the binary approach). All models employ TB loss.

Problem/ Graph	Method	Dest. Hist.	α	200			300			400			500		
				$\overline{\text{obj}}$	$\sigma(\text{obj})$	AUC[%]	$\overline{\text{obj}}$	$\sigma(\text{obj})$	AUC[%]	$\overline{\text{obj}}$	$\sigma(\text{obj})$	AUC[%]	$\overline{\text{obj}}$	$\sigma(\text{obj})$	AUC[%]
MAXCUT/ ER	ILP	–	–	1640.25	27.15	–	2470.87	41.20	–	3306.22	47.85	–	4131.78	56.13	–
	LNS-rand	–	–	1853.28	28.26	88.42	2787.07	36.43	79.97	3705.95	44.07	72.27	4604.33	54.81	64.69
	GFN-LNS	y	1.0	1858.10	26.83	94.74	2803.84	35.63	90.01	3741.30	40.41	84.53	4678.11	50.68	77.81
MAXCUT/ BA	ILP	–	–	1084.07	17.25	–	1659.57	23.08	–	2240.53	23.07	–	2814.98	26.99	–
	LNS-rand	–	–	1227.07	9.78	89.43	1867.25	11.12	80.21	2503.83	13.57	75.06	3126.55	18.84	67.70
	GFN-LNS	y	1.0	1231.05	8.61	94.59	1879.86	9.67	92.07	2528.99	14.27	87.71	3172.53	15.92	83.19
DkS/ ER	ILP	–	–	201.57	7.92	–	300.63	11.48	–	376.86	18.67	–	434.98	17.89	–
	LNS-rand	–	–	226.71	5.94	78.82	339.22	8.25	71.48	445.35	11.12	64.83	546.83	12.79	58.55
	GFN-LNS	n	0.1	238.80	6.29	94.54	366.64	7.74	92.08	493.12	10.67	88.66	618.73	11.82	84.82
DkS/ BA	ILP	–	–	665.03	25.32	–	977.32	68.74	–	1316.66	92.73	–	1623.10	97.82	–
	LNS-rand	–	–	692.25	4.15	89.48	1067.39	6.49	81.62	1428.92	8.73	77.67	1809.73	8.83	71.96
	GFN-LNS	y	0.1	704.13	3.47	96.54	1102.45	3.09	94.63	1501.65	2.82	90.77	1900.06	2.88	87.83

we adjust the edge probability for ER graphs to $p_e = 0.15$, $p_e = 0.075$, and $p_e = 0.06$ for sizes 200, 400, and 500, respectively, but keep all other graph and LNS parameters the same as in training to provide a comparable setting. Results are shown in Table 2 and illustrate that GFN-LNS is able to generalize well to different graph sizes, consistently yielding significantly better solutions than the baselines again based on one-sided paired WSRT with $p \leq 0.05$. The performance improvements even increase for larger graphs where solution sets are usually larger and a better selection of destroy sets is thus more important and beneficial. This demonstrates that the learned policies are not only effective for the graph size they were trained on but are also applicable to other sizes, which is important in practice where training individual models for each size may not be feasible.

5 Conclusion

We proposed GFlowNet-guided Large Neighborhood Search (GFN-LNS), a generic framework for learning effective destroy operators in LNS that is applicable to diverse COPs on graphs without requiring problem-specific tuning. We introduced a problem-agnostic MDP formulation for auto-regressive construction of destroy sets via GFlowNets, enabling the models to capture complex dependencies between nodes. The effectiveness of GFN-LNS is demonstrated on three challenging graph problems (MIS-BNB, MAXCUT, and DkS) across three graph types (ER, RR, and BA). In most cases, GFN-LNS improves upon both, an LNS with random destroy set selection and Gurobi on respective ILP models, in terms of final solution quality and especially online performance. Ablation studies revealed that our approach is able to learn effective policies even with minimal feature engineering, indicating robustness and generality. Notably,

the GFlowNets generalize well to different graph sizes not seen during training. On the downside, the computational overhead introduced by inferencing the GFlowNet models is significant and leads to fewer executed iterations, which can be detrimental to the performance especially when the repair operator is fast.

The lack of comparison to other learning-based approaches for selecting destroy sets within an LNS is a key limitation of our work to be addressed in the future. Several design choices such as the fixed destroy set size, the basic repair operator, and the minimalistic set of used features were intentionally kept simple to investigate the raw performance of our approach. While effective, these limitations clearly leave room for improvements. Future work could explore richer state representations and utilizing GFlowNets also for controlling other aspects of the LNS such as the destroy set size or which of a set of different repair operators should be applied.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Addanki, R., Nair, V., Alizadeh, M.: Neural large neighborhood search. In: Learning Meets Combinatorial Algorithms at NeurIPS 2020 Workshop (2020). <https://openreview.net/forum?id=xEQhKANoVW>
2. Ahuja, R.K., Ergun, Ö., Orlin, J.B., Punnen, A.P.: A survey of very large-scale neighborhood search techniques. *Discret. Appl. Math.* **123**(1–3), 75–102 (2002)
3. Bengio, E., Jain, M., Korablyov, M., Precup, D., Bengio, Y.: Flow network based generative models for non-iterative diverse candidate generation. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 27381–27394. Curran Associates, Inc. (2021)
4. Bengio, Y., Lahlou, S., Deleu, T., Hu, E.J., Tiwari, M., Bengio, E.: GFlowNet foundations. *J. Mach. Learn. Res.* **24**(1), 210:10006–210:10060 (2023)
5. Bresich, M., Raidl, G.R., Limmer, S.: Revisiting large neighborhood search with on-the-fly charging station insertion for the electric autonomous dial-a-ride problem. *ACM Trans. Evol. Learn. Optim.* (2025)
6. Chen, M., Gao, L., Chen, Q., Liu, Z.: Dynamic partial removal: a neural network heuristic for large neighborhood search. *arXiv preprint arXiv:2005.09330* (2020)
7. Cherrered, A., Oulamara, A., Cherif-Khettaf, W.R.: A deep reinforcement learning-based large neighborhood search for capacitated vehicle routing problems. *Procedia Comput. Sci.* **270**, 1568–1577 (2025)
8. Feng, S., Sun, Z., Yang, Y.: DIFUSCO-LNS: diffusion-guided large neighbourhood search for integer linear programming (2024). <https://openreview.net/forum?id=9QV7Q9gKl9>
9. Feng, S., Sun, Z., Yang, Y.: SPL-LNS: sampling-enhanced large neighborhood search for solving integer linear programs. *arXiv preprint arXiv:2508.16171* (2025)
10. Garey, M.R., Johnson, D.S.: *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman (1979)
11. Goodfellow, I.J., et al.: Generative adversarial nets. In: *Advances in Neural Information Processing Systems*, pp. 2672–2680 (2014)

12. Huang, T., Ferber, A.M., Tian, Y., Dilkina, B., Steiner, B.: Searching large neighborhoods for integer linear programs with contrastive learning. In: Proceedings of the 40th International Conference on Machine Learning, vol. 202, pp. 13869–13890 PMLR (2023)
13. Jaccard, P.: Étude comparative de la distribution florale dans une portion des alpes et du jura. *Bulletin de la Société Vaudoise des Sciences Naturelles* **37**(142), 547–579 (1901). <https://doi.org/10.5169/seals-266450>
14. Jatschka, T., Rodemann, T., Raidl, G.R.: A large neighborhood search for a cooperative optimization approach to distribute service points in mobility applications. In: *Metaheuristics and Nature Inspired Computing*, vol. 1541, pp. 3–17. Springer, Cham (2022)
15. Jiralerspong, M., et al.: Expected flow networks in stochastic environments and two-player zero-sum games. arXiv preprint [arXiv:2310.02779](https://arxiv.org/abs/2310.02779) (2024)
16. Karp, R.M.: Reducibility Among Combinatorial Problems, pp. 85–103. Plenum Press (1972)
17. Kim, M., Choi, S., Kim, H., Son, J., Park, J., Bengio, Y.: Ant colony sampling with GFlowNets for combinatorial optimization. In: Li, Y., Mandt, S., Agrawal, S., Khan, E. (eds.) *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research*, vol. 258, pp. 469–477. PMLR (2025)
18. Lamm, S., Sanders, P., Schulz, C., Strash, D., Werneck, R.F.: Finding near-optimal independent sets at scale. In: *Proceedings of the 18th Workshop on Algorithm Engineering and Experiments*, pp. 138–150. SIAM (2016)
19. Lanciano, T., Miyauchi, A., Fazzone, A., Bonchi, F.: A survey on the densest subgraph problem and its variants. *ACM Comput. Surv.* **56**(8), 1–40 (2024)
20. Limmer, S.: Bilevel large neighborhood search for the electric autonomous dial-a-ride problem. *Transp. Res. Interdisc. Perspect.* **21**, 100876 (2023)
21. Limmer, S., Varga, J., Raidl, G.R.: Large neighborhood search for electric vehicle fleet scheduling. *Energies* **16**(12) (2023)
22. Malkin, N., Jain, M., Bengio, E., Sun, C., Bengio, Y.: Trajectory balance: improved credit assignment in GFlowNets. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 5955–5967. Curran Associates, Inc. (2022)
23. Oberweger, F.F., Raidl, G.R., Rönnberg, E., Huber, M.: A learning large neighborhood search for the staff rostering problem. In: *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 300–317. Springer, Cham (2022)
24. Pan, L., Malkin, N., Zhang, D., Bengio, Y.: Better training of GFlowNets with local credit and incomplete trajectories. In: *Proceedings of the 40th International Conference on Machine Learning*, pp. 26878–26890. PMLR (2023)
25. Pan, L., Zhang, D., Jain, M., Huang, L., Bengio, Y.: Stochastic generative flow networks. In: *Proceedings of the 39th Conference on Uncertainty in Artificial Intelligence*, pp. 1628–1638. PMLR (2023)
26. Pisinger, D., Ropke, S.: Large neighborhood search. In: Gendreau, M., Potvin, J.Y. (eds.) *Handbook of Metaheuristics*, pp. 399–419. Springer, Cham (2010)
27. Shaw, P.: Using constraint programming and local search methods to solve vehicle routing problems. In: *International Conference on Principles and Practice of Constraint Programming*, pp. 417–431. Springer, Cham (1998)
28. Song, J., Lanka, R., Yue, Y., Dilkina, B.: A general large neighborhood search framework for solving integer linear programs. *Adv. Neural. Inf. Process. Syst.* **33**, 20012–20023 (2020)

29. Sonnerat, N., Wang, P., Ktena, I., Bartunov, S., Nair, V.: Learning a large neighborhood search algorithm for mixed integer programs. arXiv preprint [arXiv:2107.10201](https://arxiv.org/abs/2107.10201) (2021)
30. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press (2018)
31. van Bevern, R., Fluschnik, T., Mertzios, G.B., Molter, H., Sorge, M., Suchý, O.: The parameterized complexity of finding secluded solutions to some classical optimization problems on graphs. *Discret. Optim.* **30**, 20–50 (2018)
32. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? In: International Conference on Learning Representations (2019). <https://openreview.net/forum?id=ryGs6iA5Km>
33. Zhang, D.W., Rainone, C., Peschl, M., Bondesan, R.: Robust scheduling with GFlowNets. arXiv preprint [arXiv:2302.05446](https://arxiv.org/abs/2302.05446) (2023)
34. Zhang, D., Dai, H., Malkin, N., Courville, A., Bengio, Y., Pan, L.: Let the flows tell: solving graph combinatorial optimization problems with GFlowNets. In: Proceedings of the 37th International Conference on Neural Information Processing Systems, pp. 11952–11969. Curran Associates, Inc. (2023)
35. Zhang, N., Yang, J., Cao, Z., Chi, X.: Adversarial generative flow network for solving vehicle routing problems. In: International Conference on Learning Representations (2025). <https://openreview.net/forum?id=tBom4xOW1H>